

Contents

1	DD02: プロジェクト・FAQ 管理	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	2
1.4.1	3.1 画面詳細設計（参照のみ）	2
1.4.2	3.2 機能詳細設計（参照のみ）	2
1.4.3	3.3 データベース詳細設計（参照のみ）	2
1.4.4	3.4 実装モジュール構成	2
1.4.5	3.5 FAQ 公開ガード（AC-006）	2
1.4.6	3.6 FTS5 検索	2
1.4.7	3.7 埋め込みベクトル事前計算	3
1.4.8	3.8 一括インポート（faq.bulk_import）	3
1.4.9	3.9 リミット	3
1.4.10	3.10 プロジェクト単位 IP 許可リスト（FR-179 / FR-330）	3
1.4.11	3.11 プロジェクト作成時の管理者必須指定（FR-015e / FR-030a）	3
1.4.12	3.12 プロジェクト削除時の孤立メンバー cleanup（FR-030b）	4
1.4.13	3.13 関連する横断設計	4
1.5	4. 関連設計	4
1.6	5. テスト観点	4
1.6.1	5.1 その他観点	5
1.7	6. 未確定事項・確認事項	5

1 DD02: プロジェクト・FAQ 管理

1.1 0. 文書情報

項目	内容
文書名	DD02: プロジェクト・FAQ 管理
詳細設計 ID	DD02
対象システム	FAQ AI ウィジェット SaaS / メインシステム
関連機能 ID	FR-040～048（FAQ CRUD / 公開ガード） / FR-100～103, FR-105, FR-106（未解決質問 → FAQ 登録）
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-040～048	FAQ CRUD / 自動公開禁止 / 公開・取下 / 一括インポート
機能	FR-100～103, FR-105, FR-106	未解決質問 → FAQ 登録（初期反映 / 確認・編集 / 登録元トレース）
画面	SCR-010	プロジェクト一覧
画面	SCR-011	FAQ 一覧
画面	SCR-012	FAQ 編集
API	/projects/* / faqs/*	プロジェクト・FAQ 操作
テーブル	projects faqs faq_embeddings faq_search_fts	FAQ 基盤

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 6 SCR-010～012	画面詳細設計	プロジェクト一覧 / FAQ 一覧 / FAQ 編集（正本は基本設計）
§ 7	機能詳細設計（FAQ 関連）	FAQ CRUD / 公開ガード / 一括インポート（正本は基本設計）
§ 9	データベース詳細設計	projects / faqs テーブル詳細（正本は基本設計）
§ 3.3.1	worker-main-api モジュール構成	routes/projects.ts routes/faqs.ts

1.4 3. 詳細設計本文

1.4.1 3.1 画面詳細設計（参照のみ）

SCR-010 プロジェクト一覧 / SCR-011 FAQ 一覧 / SCR-012 FAQ 編集の画面詳細は [../02_基本設計/01_画面設計.md](#) を正本とする。メッセージ ID は [../02_基本設計/06_メッセージ一覧.md](#) を参照。

1.4.2 3.2 機能詳細設計（参照のみ）

FAQ CRUD / 公開ガード / 一括インポートの全項目は [../02_基本設計/02_API 設計.md](#) を正本とする。

1.4.3 3.3 データベース詳細設計（参照のみ）

projects / faqs / faq_embeddings / faq_search_fts の DDL、CHECK 制約、インデックス、外部キー、コード値、SaaS データ分離観点（owner_account_id）、保持期間は [../02_基本設計/03_テーブル設計.md](#) を正本とする。

1.4.4 3.4 実装モジュール構成

```
src/
├── routes/
│   ├── projects.ts      # /projects/* CRUD + メンバー割当
│   └── faqs.ts          # /faqs/* CRUD + 公開 / 取下 / 一括インポート / 提案承認
├── handlers/
├── domain/
│   └── faq-status.ts    # FAQ 状態遷移 (draft / review / published / unpublished)
├── repository/
│   ├── projects.ts
│   ├── faqs.ts
│   └── faq-embeddings.ts
├── middleware/
│   └── authorize.ts     # requireProject / requireRole 連動
```

1.4.5 3.5 FAQ 公開ガード（AC-006）

faqs.status は CHECK 制約で値域固定。status='published' への遷移は人手承認 API（POST /api/v1/faqs/{id}/publish）のみ許可。domain/faq-status.ts の canTransition() で draft → published を拒否、review → published のみ許可する。

1.4.6 3.6 FTS5 検索

faq_search_fts は SQLite の FTS5 virtual table で構築し、faqs.title || faqs.body をインデックス対象とする。FAQ 公開・更新時に同期更新。検索クエリは BM25 ランキングを利用：

```
SELECT f.*, bm25(faq_search_fts) AS rank
FROM faq_search_fts JOIN faqs f ON f.rowid = faq_search_fts.rowid
WHERE faq_search_fts MATCH ?1 AND f.project_id = ?2 AND f.status = 'published'
ORDER BY rank LIMIT 20
```

詳細は [DD03_AI 回答パイプライン.md](#) § 3.2 関連度計算を参照。

1.4.7 3.7 埋め込みベクトル事前計算

FAQ 公開時に @cf/baai/bge-base-en-v1.5 で埋め込みベクトルを計算し、faq_embeddings テーブル（または KV embedding:{faqId}）に保存。AI 推論時のリランキング用。

1.4.8 3.8 一括インポート (faq.bulk_import)

CSV/JSON ファイル → R2 ステージング → Queue 投入 → consumer が分割 INSERT。1 リクエスト最大 1000 件。詳細は [DD14_ バッチ・非同期処理.md](#) を参照。

1.4.9 3.9 リミット

項目	警告	拒否
FAQ 件数 / 契約	8,000	12,000 (409 FAQ_LIMIT_EXCEEDED)
プロジェクト数 / 契約	40	50 (409 PROJECT_LIMIT_EXCEEDED)
IP 許可 CIDR 数 / プロジェクト	-	100 (400 E-BIZ-IP-002、SCR-010-M1 で行番号付きエラー)

1.4.10 3.10 プロジェクト単位 IP 許可リスト (FR-179 / FR-330)

- データ: project_ip_allowlist ([03_テーブル設計.md § 3.8a](#) 参照)
- API: PATCH /projects/{id} のフィールド ipAllowlist、または PATCH /projects/{id}/ip-allowlist ([02_API 設計.md § 5.3.3 / § 5.3.3a](#) 参照)
- 評価ロジックの正本: [02_API 設計.md § 5.5.0](#)
- キャッシュ: ウィジェット Worker は KV: ip_allowlist:{projectId} に CIDR セットを 5 分 TTL で保持し、LPM 判定はメモリ上で実施。PATCH 成功時に該当キーを即時 invalidate
- 監査ログ: project_ip_allowlist.update (metadata に変更前後の CIDR セット差分、retention_class=general)
- 自己検証ガード: API 層で各行を ipaddr ライブラリでパースし、IPv4Network/IPv6Network のいずれかでなければ 400。重複は事前にセット化して検出。CIDR の正規化 (203.0.113.0/24 形式) を保存前に実施

1.4.11 3.11 プロジェクト作成時の管理者必須指定 (FR-015e / FR-030a)

POST /projects 実行時は SCR-010-M1(新規作成モード)からの initialAdmins を受け取り、以下のトランザクション境界で処理する:

- リクエスト検証: selfAsAdmin=false の場合は invitees[].role='admin' が 1 件以上必須。違反は 400 VALIDATION_ERROR(field: "initialAdmins")。invitees[].email の同一オーナー配下重複 / リスト内重複は 409 / 400。
- projects INSERT(ウィジェット公開 を発行)
- invitees[] の各行について:
 - 既存メンバーアカウントが存在する場合 (同一オーナー配下 + 同一 email_hmac): account_project_grants に当該プロジェクト × 指定ロールを INSERT のみ + 通知メール
 - 新規メールアドレスの場合: accounts を status='pending_activation' で作成 + account_project_grants を同時 INSERT + access_tokens.purpose='activation'(有効期限 7 日) 発行 + 招待メール送信 Queue 投入
- selfAsAdmin=true のときはオーナー側 account_project_grants の追加処理は 不要 (オーナーは is_owner=1 判定で全プロジェクト bypass)。論理上「オーナーが当該プロジェクトの管理者として振る舞う」ことを initialAdmins.selfAsAdmin フラグの記録のみで担保する
- 監査ログ project.created_with_initial_admins(metadata に { projectId, ownerAsAdmin, invitees: [{ email_hmac, role }] }, retention_class=general) を記録

擬似コード:

```
async function createProject(actor: Principal, req: CreateProjectRequest) {
  requireOwner(actor); // E-AUTHZ-OWNER-ONLY
  validateInitialAdmins(req.initialAdmins); // 400 / 409
  await db.transaction(async (tx) => {
    const project = await tx.insert('projects', { ...req, owner_account_id: actor.ownerAccountId })
    for (const invitee of req.initialAdmins.invitees) {
      const existing = await tx.findMemberByEmailHmac(actor.ownerAccountId, hmac(invitee.email))
      if (existing) {
```

```

    await tx.insert('account_project_grants', { account_id: existing.id, project_id: project_id });
  } else {
    const acc = await tx.insert('accounts', { owner_account_id: actor.ownerAccountId, email: actor.email });
    await tx.insert('account_project_grants', { account_id: acc.id, project_id: project.id });
    const token = await tx.insert('access_tokens', { account_id: acc.id, purpose: 'activation' });
    await emailQueue.enqueue({ to: invitee.email, template: 'TPL-ADMIN_USER_REGISTER', token });
  }
}
await tx.insertAudit({ action: 'project.created_with_initial_admins', target_type: 'project' });
return project;
});
}

```

1.4.12 3.12 プロジェクト削除時の孤立メンバー cleanup(FR-030b)

DELETE /projects/{id} 実行時は以下のトランザクション境界で処理する:

1. 削除前のスナップショット: 当該プロジェクトに割当のあるメンバー accountId 一覧を取得
2. account_project_grants から当該 project_id の全行を DELETE
3. 孤立メンバーの抽出: 「ステップ1のメンバーのうち、他プロジェクト割当が0件かつ is_owner=0 のもの」
4. 孤立メンバーの全セッションを失効(sessions.revoked_atを設定)+ accounts から物理削除
5. プロジェクト本体および紐づくデータ (FAQ / 質問ログ / 案件 / チャット 等) を削除 / 匿名化 (削除モードに従う)
6. 監査ログ project.deleted_with_orphan_cleanup(metadata に { projectId, deletedAccountIds }, retention_class=general) を記録

オーナー (is_owner=1) は本処理の対象外 (常に存続)。孤立判定は DDL の ON DELETE CASCADE では実現できないため、必ずアプリ層のトランザクション内で実施する。

1.4.13 3.13 関連する横断設計

- ・認可: DD09_認可ヘルパ.md の requireProjectRole でオーナー境界 + プロジェクトロール (admin / member) を検証
- ・監査ログ: faq.create / faq.update / faq.publish / faq.unpublish / faq.bulk_import / project.created_with_initial_admins / project.deleted_with_orphan_cleanup / project.member_invited を retention_class=general で記録
- ・リアルタイム反映: FAQ 公開・編集時に widget-api 側の KV キャッシュ (embedding:{faqId}、ai_threshold:{owner}:{project} は別系統) を invalidate

1.5 4. 関連設計

種別	参照先
要件	../01_要件定義/index.md
基本設計	../02_基本設計/index.md
画面設計	../02_基本設計/01_画面設計.md
テーブル設計	../02_基本設計/03_テーブル設計.md
API 設計	../02_基本設計/02_API 設計.md
運用設計	../04_運用設計/index.md
将来対応	../05_future/index.md
関連 DD	DD03_AI 回答パイプライン.md / DD09_認可ヘルパ.md / DD10_監査ログ書込・完全性検証.md / DD13_ウィジェット配信.md

1.6 5. テスト観点

AC ID	テスト ID	テスト方式	テストファイル
AC-006	u-faq-publish-001 / it-faq-publish-001	Unit + Integration	workers/main-api/test/{unit,integration}/faq/publish-guard.test.ts

AC ID	テスト ID	テスト方式	テストファイル
AC-014	e2e-faq-crud-001	E2E	apps/admin/e2e/faq/crud.spec.ts

1.6.1 5.1 その他観点

観点	内容
単体	faq-status.ts 遷移ガード全パターン
結合	FTS5 検索の MATCH 構文 / BM25 ランキング
異常系	他オーナーのプロジェクトに対する FAQ CRUD アクセス時 404
境界値	FAQ 件数 7999 / 8000 / 8001 / 11999 / 12000 / 12001
権限	該当プロジェクトに割当のないメンバー (account_project_grants 行なし) による FAQ 編集試行で 404 偽装 / 該当 PJ の member+ ロールを持つメンバーは編集可
性能	POST /api/v1/faqs/{id}/publish p95 < 300ms (UPDATE 1 行 + KV invalidate)

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済